

Exercícios — Aula 08 — Recursão em arrays e busca binária recursiva

Técnicas de Programação - 2026/02

Última atualização: October 5, 2026

Instruções

- Em recursão com arrays, o índice ou intervalo faz parte do estado.
- Nas simulações, registre parâmetros da chamada, caso base, retorno e chamadas pendentes.
- Para busca binária recursiva, mantenha a pré-condição: o array está ordenado.

Exercício 1 — Soma recursiva em array

Simule o algoritmo para $v = [4, 7, 2]$ e $i = 0$.

Algorithm 1: SomaAPartir

Result: Executa SOMA-A-PARTIR.

Input : array v , int i

Output: int

```
if  $i = TAMANHO(v)$  then
|   return 0
end
return  $v[i] + SOMA-A-PARTIR(v, i + 1)$ 
```

Preencha a tabela de chamadas.

chamada	valor de i	expressão pendente	próxima chamada
SOMA-A-PARTIR(v , 0)	0	$4 + \text{SOMA-A-PARTIR}(v, 1)$	
SOMA-A-PARTIR(v , 1)			
SOMA-A-PARTIR(v , 2)			
SOMA-A-PARTIR(v , 3)			

Preencha a tabela de retornos.

retorno de	valor retornado
SOMA-A-PARTIR(v , 3)	
SOMA-A-PARTIR(v , 2)	
SOMA-A-PARTIR(v , 1)	
SOMA-A-PARTIR(v , 0)	

Qual é o resultado de SOMA-A-PARTIR(v , 0)?

Exercício 2 — Busca linear recursiva

Escreva pseudocódigo para BUSCA-LINEAR-RECURSIVA.

Contrato:

- entrada: array `v`, valor `alvo`;
- saída: último índice em que `alvo` aparece;
- se o `alvo` não aparece, retorne `-1`;
- use um algoritmo auxiliar com o índice atual.

Esqueleto sugerido:

Algorithm 2: BuscaLinearRecursiva

Result: Executa BUSCA-LINEAR-RECURSIVA.

Input : array `v`, value `alvo`

Output: int

return *BUSCA-A-PARTIR*(*v*, *alvo*, *v.length* - 1)

Algorithm 3: BuscaAPartir

Result: Executa BUSCA-A-PARTIR.

Input : array `v`, value `alvo`, int `i`

Output: int

...

Teste com:

- `v` = [8, 3, 5, 3], `alvo` = 8;
- `v` = [8, 3, 5, 3], `alvo` = 3;
- `v` = [8, 3, 5, 3], `alvo` = 9;
- `v` = [], `alvo` = 8.

Exercício 3 — Busca binária recursiva

Simule a busca por 21 em:

$v = [2, 5, 8, 12, 16, 21, 27]$

Algorithm 4: BuscaBinariaRecursiva

Result: Executa BUSCA-BINARIA-RECURSIVA.

Input : array ordenado v , value alvo

Output: índice do alvo ou -1

return *BUSCA-INTERVALO*(v , $alvo$, 0 , *TAMANHO*(v) - 1)

Algorithm 5: BuscaIntervalo

Result: Executa BUSCA-INTERVALO.

Input : array ordenado v , value alvo, int inicio, int fim

Output: índice do alvo ou -1

if $inicio > fim$ **then**

return -1

end

$meio \leftarrow inicio + \text{INTEIRO}((fim - inicio)/2)$

if $v[meio] = alvo$ **then**

return $meio$

end

if $v[meio] < alvo$ **then**

return *BUSCA-INTERVALO*(v , $alvo$, $meio + 1$, fim)

end

return *BUSCA-INTERVALO*(v , $alvo$, $inicio$, $meio - 1$)

chamada	inicio	fim	meio	$v[meio]$	próxima chamada ou retorno
1					
2					
3					

Agora simule para $alvo = 6$ e indique em que chamada o caso base é atingido.

Exercício 4 — Completar auxiliar de contagem

Complete o pseudocódigo para contar quantas vezes **alvo** aparece no array.

Algorithm 6: ContarOcorrencias

Result: Executa CONTAR-OCORRENCIAS.

Input : array v , value $alvo$

Output: int

return *CONTAR-A-PARTIR*(v , $alvo$, 0)

Algorithm 7: ContarAPartir

Result: Executa CONTAR-A-PARTIR.

Input : array v , value $alvo$, int i

Output: int

```

if _____ then
    | return _____
end
resto ← CONTAR-A-PARTIR( $v$ ,  $alvo$ , _____)
if _____ then
    | return _____
end
return _____

```

Teste com:

- $v = [1, 2, 1, 3, 1]$, $alvo = 1$;
- $v = [1, 2, 1, 3, 1]$, $alvo = 4$;
- $v = []$, $alvo = 1$.

Exercício 5 — Pilha de chamadas: linear versus binária

Considere um array ordenado de tamanho 16.

1. No pior caso, quantas chamadas a busca linear recursiva pode fazer?
2. No pior caso, quantas chamadas a busca binária recursiva faz aproximadamente?
3. Qual delas tem pilha de chamadas com profundidade $O(n)$?
4. Qual delas tem pilha de chamadas com profundidade $O(\log n)$?
5. Se o array dobrar de tamanho, o que acontece com a quantidade de chamadas de cada uma?

Preencha a tabela com estimativas.

tamanho n	busca linear recursiva no pior caso	busca binária recursiva no pior caso
8		
16		
32		
64		

Exercício 6 — Caso base incorreto

O algoritmo abaixo tenta somar um array recursivamente, mas tem um caso base perigoso.

Algorithm 8: SomaAPartirComErro

Result: Executa *SOMA-A-PARTIR-COM-ERRO*.**Input** : array v , int i **Output:** int

```
if  $i = TAMANHO(v) - 1$  then
    return 0
end
return  $v[i] + SOMA-A-PARTIR-COM-ERRO(v, i + 1)$ 
```

1. Simule para $v = [4, 7, 2]$, começando em $i = 0$.
2. Qual elemento deixa de ser somado?
3. O que acontece se $v = []$ e começamos em $i = 0$?
4. Reescreva o caso base corretamente.
5. Reescreva a linha de retorno se o caso base correto for usado.